PID-RS TECHNICAL REPORT SERIES

Exact Multiplicative Certificates for Empirical Categorical SxPID2

A complete sign reduction, a retained counterexample, and bounded executable refinement evidence

pid-rs project analysis

25 July 2026

Abstract

This note gives a self-contained exact-arithmetic assurance route for the complete averaged two-source empirical categorical shared-exclusions PID encoded by the pid-rs definition revision `makkeh-gutknecht-wibral-2021-empirical-sxpid2-v1`. After the published keyed source-event disjunctions, keyed target intersections, empirical averaging rule, and integer Möbius transform are fixed, every cumulative and atom is exactly $n^{-1} \log R$ for a positive rational product R. Hence its exact zero and sign are decided without a transcendental approximation by comparing R with one.

The first implementation used an empty canonical log-term map as its only exact-zero witness. That witness is sound but incomplete. An exhaustive binary search produced a minimized five-term, total-count-eight counterexample whose exact product is one although its canonical term map is nonempty. The live directed interval correctly straddles zero. The repaired schema keeps that interval decision unresolved and emits a separate bounded exact-product zero certificate. The proof, the failure, the repair, resource preflight, independent verification, Lean kernel check, exhaustive boundary, mutations, and scientific claim boundary are all stated explicitly. No new PID is defined, and no claim about population calibration, continuous PID, or downstream authority follows.

Contents

1 Scope and nonclaims

The mathematical statement is representation-agnostic only after one representation has already been fixed: the Makkeh–Gutknecht–Wibral shared-exclusions event construction, its two-source redundancy lattice, exact empirical probabilities, and its integer Möbius transform. Changing the event logic, lattice, smoothing rule, or weights changes the certified object. Exact arithmetic does not establish that SxPID is the unique, universally preferred, or scientifically adequate PID.

The result applies to one finite empirical count table. It does not prove data authenticity, population support, independence, ergodicity, stationarity, dependence-color premises, drift bounds, estimator consistency, calibrated uncertainty, equivalence of quantized and continuous estimands, continuous KSG or ISX validity, higher-source PID, or downstream operational fitness. It does not prove pid-core binary64 results. It also does not verify the Rust compiler, Python, GMP, MPFR, Rug, hardware, or the operating system.

2 Fixed empirical definition

2.1 Count table and keyed events

Let Z_+ be the finite set of observed complete states $z = (s_1, s_2, t)$. Each state has an integer multiplicity $c_z > 0$, and

$$n = \sum_{z \in Z_+} c_z > 0. \tag{1}$$

Fix a two-source redundancy-lattice node α. For a keyed observed state z, let $A_{z,\alpha}$ be the disjunction of the node's source-collection events, exactly as in the shared-exclusions construction. Define integer masses

$$a_{z,\alpha} = \#A_{z,\alpha}, \tag{2}$$
$$b_{z,\alpha} = \#(A_{z,\alpha} \cap \{T = t_z\}), \tag{3}$$
$$t_z = \#\{T = t_z\}. \tag{4}$$

Every defining event contains its keyed observed state, so

$$0 < c_z \le b_{z,\alpha} \le a_{z,\alpha} \le n, \qquad b_{z,\alpha} \le t_z \le n. \tag{5}$$

Thus every rational logarithm argument below is strictly positive.

2.2 Pointwise and averaged cumulatives

In nats, define

$$i^{+}_{z,\alpha} = \log \frac{n}{a_{z,\alpha}}, \tag{6}$$

$$i^{-}_{z,\alpha} = \log \frac{t_z}{b_{z,\alpha}}, \tag{7}$$

$$i^{\mathrm{sx}}_{z,\alpha} = i^{+}_{z,\alpha} - i^{-}_{z,\alpha} = \log \frac{n b_{z,\alpha}}{a_{z,\alpha} t_z}. \tag{8}$$

For $u \in \{+, -, \mathrm{sx}\}$, the empirical averaged cumulative is

$$C^{u}_{\alpha} = \sum_{z \in Z_+} \frac{c_z}{n} i^{u}_{z,\alpha}. \tag{9}$$

No pseudo-count, smoothing rule, or alternate weighting is silently inserted.

2.3 Two-source atoms

Let $M_{\gamma\alpha} \in \mathbb{Z}$ be the fixed Möbius coefficient mapping cumulatives to an atom γ. The informative, misinformative, and net atoms are

$$\Pi^{u}_{\gamma} = \sum_{\alpha} M_{\gamma\alpha} C^{u}_{\alpha}. \tag{10}$$

Integer coefficients are the essential algebraic property. Negative coefficients correspond to exact reciprocals in the product representation.

3 Exact product normalization

Definition 3.1 (Cumulative products)**.** For each node α, define positive rationals

$$R^{+}_{\alpha} = \prod_{z \in Z_+} \left(\frac{n}{a_{z,\alpha}} \right)^{c_z}, \tag{11}$$

$$R^{-}_{\alpha} = \prod_{z \in Z_+} \left(\frac{t_z}{b_{z,\alpha}} \right)^{c_z}, \tag{12}$$

$$R^{\mathrm{sx}}_{\alpha} = \prod_{z \in Z_+} \left(\frac{n b_{z,\alpha}}{a_{z,\alpha} t_z} \right)^{c_z}. \tag{13}$$

Theorem 3.2 (Exact cumulative normalization)**.** *For every fixed node and component,*

$$C^{u}_{\alpha} = \frac{1}{n} \log R^{u}_{\alpha}. \tag{14}$$

Moreover,

$$R^{\mathrm{sx}}_{\alpha} = \frac{R^{+}_{\alpha}}{R^{-}_{\alpha}}. \tag{15}$$

Proof. Substitute equations (6)–(8) into equation (9). For the informative component,

$$C^{+}_{\alpha} = \frac{1}{n} \sum_{z} c_z \log \frac{n}{a_{z,\alpha}} = \frac{1}{n} \log \prod_{z} \left(\frac{n}{a_{z,\alpha}} \right)^{c_z} = \frac{1}{n} \log R^{+}_{\alpha}.$$

The same logarithm product and integer-power rules prove the misinformative and net identities. Equation (8) gives the local ratio of informative to misinformative arguments. Raising each ratio to c_z, multiplying, and using equations (11)–(13) proves equation (15). □

Definition 3.3 (Atom products)**.** For every atom γ, let

$$R^u_\gamma = \prod_\alpha (R^u_\alpha)^{M_{\gamma\alpha}}. \tag{16}$$

Theorem 3.4 (Exact atom normalization)**.** *Every two-source atom satisfies*

$$\Pi^u_\gamma = \frac{1}{n} \log R^u_\gamma. \tag{17}$$

Proof. Apply Theorem 3.2 in equation (10):

$$\Pi^u_\gamma = \frac{1}{n} \sum_\alpha M_{\gamma\alpha} \log R^u_\alpha = \frac{1}{n} \log \prod_\alpha (R^u_\alpha)^{M_{\gamma\alpha}} = \frac{1}{n} \log R^u_\gamma.$$

The signed integer power rule is valid because every R^u_α is positive. □

Corollary 3.5 (Complete exact sign decision)**.** *For any cumulative or atom represented by a positive rational R,*

$$\frac{1}{n} \log R = 0 \iff R = 1, \tag{18}$$

$$\frac{1}{n} \log R > 0 \iff R > 1, \tag{19}$$

$$\frac{1}{n} \log R < 0 \iff R < 1. \tag{20}$$

Proof. Equation (1) implies $1/n > 0$, so multiplication by $1/n$ preserves order and zero. The natural logarithm is strictly increasing on $(0, \infty)$, and $\log 1 = 0$. Therefore its zero, positive, and negative regions are respectively $R = 1$, $R > 1$, and $0 < R < 1$. □

Remark 3.6 (Canonical log-linear form)**.** If a report coordinate is $V = \sum_j q_j \log x_j$, empirical construction gives $nq_j \in \mathbb{Z}$. Hence

$$V = \frac{1}{n} \log R, \qquad R = \prod_j x_j^{nq_j}. \tag{21}$$

This binds the event-count product and emitted term-list product. It is not a prime-factorization claim.

4 Retained failure: nonempty terms can cancel exactly

Counterexample 4.1 (The empty-term witness is incomplete)**.** Use canonical binary state order

$$(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)$$

and counts

$$(0,0,1,1,1,4,1,0), \qquad n = 8. \tag{22}$$

For the net unique-one atom, exact extraction gives the nonempty expression

$$E = -\frac{1}{8} \log \frac{8}{15} + \frac{1}{8} \log \frac{4}{5} + \frac{1}{8} \log \frac{8}{9} + \frac{1}{8} \log \frac{4}{3} - \frac{1}{8} \log \frac{16}{9}. \tag{23}$$

Its denominator-cleared product is

$$\begin{aligned} R &= \left(\frac{8}{15}\right)^{-1} \left(\frac{4}{5}\right) \left(\frac{8}{9}\right) \left(\frac{4}{3}\right) \left(\frac{16}{9}\right)^{-1} \\ &= \frac{15}{8} \frac{4}{5} \frac{8}{9} \frac{4}{3} \frac{9}{16} = 1. \end{aligned} \tag{24}$$

Therefore $E = \frac{1}{8} \log R = 0$, despite five retained canonical terms.

At 256 working bits, the live directed interval is

$$[-3\,2^{-258},\ 5\,2^{-259}]. \tag{25}$$

It correctly contains zero and correctly fails to determine a strict sign. The old rule "exact zero only if the canonical map is empty" therefore missed a true exact zero. Replacing the interval decision by exact zero would also be wrong, because the interval endpoints do not prove it. Report schema v2 preserves both truths:

- interval decision: `unresolved_sign`, interval zero witness absent;
- exact-product decision: `certified_exact_zero`;
- exact-product witness: `exact_multiplicative_product_equals_one`.

Exhaustive weak-composition enumeration covers every nonzero binary table with total count $1 \le n \le 8$: 12,869 tables and 308,856 coordinates. No nonempty product-one expression occurs below $n = 8$. Exactly 16 occur at $n = 8$; all have support size five, five canonical terms, and are net unique-one or net unique-two atoms. Thus $n = 8$ and support five are minimal only inside this exhausted binary domain.

5 Bounded executable refinement

5.1 Preflight before powering

For each canonical term, the Rust route first checks that $e_j = nq_j$ is a nonzero integer. It then computes without exponentiation

$$B = \sum_j |e_j|(\mathrm{bits}(\mathrm{num}\, x_j) + \mathrm{bits}(\mathrm{den}\, x_j)). \tag{26}$$

The route admits exact powers only under all ceilings in Table 1.

Table 1: Exact-product resource policy.

Limit	Ceiling
Canonical terms in one compared expression	256
Absolute denominator-cleared exponent	16,384
Projected product bits in one expression	262,144
Aggregate projected bits over admitted coordinates	1,048,576
Parser total-count bit ceiling (not a powering permission)	8,192

The preflight uses arbitrary-precision integers only for exponent and projection calculations. An expression over a per-expression limit records `not_compared_per_expression_preflight_limit`. If individually admissible expressions exceed the aggregate ceiling, nonempty expressions record `not_compared_total_preflight_limit`. These are fallbacks, not rejected interval certificates. No rational power is allocated for an unavailable plan.

The independent report-term checker uses the same plan/evaluate order. It parses bounded rationals, computes every local projection, and fixes the aggregate admission result before it calls its rational-power primitive. Two sentinel controls replace that primitive with a function that fails on any call, then exercise a locally rejected plan and an aggregate-rejected plan. Both guards reject with zero power calls. This is executable ordering evidence, not a formal time or space theorem for either runtime.

5.2 Exact comparison and interval consistency

For an admitted plan, the producer computes exact positive rational powers and multiplies them as normalized rationals. It compares the final numerator and denominator. It does not serialize a potentially large product and does not factor integers. A postcondition checks that the final numerator-plus-denominator bit length does not exceed equation (26).

The exact-product and directed-interval results have different semantics. For an interval $I = [L, U]$, consistency requires

$$R > 1 \Longrightarrow U > 0, \qquad R < 1 \Longrightarrow L < 0, \qquad R = 1 \Longrightarrow L \leq 0 \leq U.$$

The inequalities for strict product signs are themselves strict: an interval ending at zero cannot enclose a positive value, and an interval beginning at zero cannot enclose a negative value. An exact result may refine an unresolved interval, but it never changes the interval-local field.

5.3 Independent verifier

The standard-library Python verifier independently reconstructs all keyed event counts, exact log-linear expressions, the integer Möbius transform, and the product preflight. For an admitted coordinate it independently evaluates the rational powers using `Fraction` and validates the report decision and witness. Independently of that sign path, it proves each directed interval contains the exact-real logarithmic expression using outward integer fixed-point bounds on

$$\log y = 2 \sum_{k \geq 0} \frac{z^{2k+1}}{2k+1}, \qquad z = \frac{y-1}{y+1}, \quad 1 \leq y < 2,$$

with $0 \leq z \leq 1/3$ and tail bound

$$0 \leq R_m \leq \frac{2z^{2m+1}}{(2m+1)(1-z^2)} \leq \frac{9z^{2m+1}}{4(2m+1)}. \tag{27}$$

The two implementations share the mathematical specification but not the arithmetic library. Python and this verifier remain trusted; this is not a formal refinement theorem.

6 Formal and adversarial evidence

6.1 Lean kernel check

The current pinned Lean 4.33.0 project checks a standalone module proving seven results: finite signed-power log normalization, the scaled $n^{-1} \log R$ form, positive, negative, and zero equivalences, the explicit reciprocal cancellation

$$\log x + \log(x^{-1}) = 0 \qquad (x > 0,\ x \neq 1). \tag{28}$$

and the exact rational identity for all five factors in equation (24). The checker rejects `sorry`, `admit`, `axiom`, and `unsafe`. The reported axiom basis is `propext`, `Classical.choice`, and `Quot.sound`. Lean does not yet formalize the concrete SxPID events, count extractor, lattice table, Rust refinement, or resource behavior. The independent exact-rational and Rust routes bind the checked five-factor identity to the SxPID2 unique-one net atom. The current execution receipt is the versioned `sxpid2-exact-product-lean-check-4.33.0.json`. The unversioned receipt is retained byte-for-byte as historical Lean 4.32 evidence and is not credited as a current replay or compared to the current checker as if contemporary.

6.1.1 Checker qualification, terms, and provenance

The *production checker* is the fixed Python program `scripts/check-lean-exact-log-product.py`, with SHA-256 `52510a18ac5fa8b94113bfeba84f61cb28bdbe56be278fc76fb4d55407cb2dcd`. It first requires the exact 5,357-byte Lean source with SHA-256 `f0727ea3061d561ba89ba49edebece971ce03bdecf03e0c32774a1c080dc07bf`. It rejects the raw word tokens `sorry`, `admit`, `axiom`, and `unsafe`. It then invokes the pinned Lean 4.33.0 release, compiles the source plus seven `#print axioms` queries, and requires the exact permitted-axiom line for every named theorem.

A *hostile mutation* is one deliberate change to a temporary source copy. A *test-only digest rebind* replaces the expected source digest inside the self-test process so that changed bytes can reach a later policy or kernel check. Production CI never rebinds that digest. A *negative control* is a deliberately bad input that must be rejected. A *positive control* is a deliberately valid input that must be accepted. A *scope probe* is an accepted input that exposes what the named audit does not inspect. An accepted *known limitation* receives no positive-proof or mutation-kill credit.

6.1.2 Exact objects and threat model

The qualification keeps nine objects separate.

1. The *Lean theorem source* states the seven generic results sent to Lean. It does not contain the concrete SxPID event extractor.

2. The *production checker* is the unchanged Python gate. It binds the theorem-source digest, pinned project files, seven qualified names, and permitted axiom list, then runs Lean.

3. The *pinned Lean project* is `lean-toolchain`, `lakefile.toml`, and `lake-manifest.json`. It selects Lean 4.33.0 and the reviewed package closure; its hashes do not prove that an executable was built from reviewed source.

4. The *query file* temporarily concatenates the theorem source and seven `#print axioms` commands. Lean compiles it; it is not a new retained theorem source.

5. The *production receipt* is one successful checker's canonical JSON. It records exact identities and the boundary, but is execution evidence rather than an authenticity certificate.

6. A *mutant* is one temporary source copy with one declared change and its own digest. Its result applies only to that change, not every possible fault.

7. The *hostile self-test* separately loads captured production-checker bytes, performs mutants and probes, restores globals, and emits one canonical record. It is not a replacement checker.

8. The *hostile evidence JSON* is the byte-identical output of normal and optimized hostile runs. It records cases and boundaries; it does not add a theorem.

9. The *archive adjudication memo* preserves the rejected design and counterexample. It is negative engineering evidence with no production authority.

The threat model addresses accidental drift and bounded ordinary-process interference: changed checker or source bytes, symbolic-link and extra-hard-link leaves, parent replacement during one captured read, mid-read byte or metadata changes, ambient Python packages and bytecode writes, optimization-sensitive controls, ambiguous JSON, merged output streams, leaked test globals, semantic theorem changes, missing names, and added axiom dependencies. The character-literal witness challenges the discarded partial lexer; the extra-declaration probes challenge the scope of the seven-name query.

The model does not claim protection from a privileged or same-user adversary that coordinates changes between bounded observations, forges the Python or Lean environment, subverts the kernel, compiler, dynamic loader, filesystem, SHA-256 implementation, hardware, or absent history. SHA-256 proves equality to reviewed bytes, not origin, authorship, trusted time, or authenticity. Self-created mutants live in a private temporary directory and assume no concurrent writer. The stable captured-read protocol applies to the tracked checker, theorem source, and hostile harness. It is an endpoint comparison, not an atomic snapshot or sandbox.

The replacement hostile suite is project-defined engineering evidence. It followed review of discarded archive commit `6077443`. That draft removed comments and strings with a handwritten scanner that did not model character literals. Two quote-valued `Char` literals around an empty string made a later live, unqueried `axiom` invisible to the scanner. The complete 5,492-byte witness has SHA-256 `891323ef49a0a9e2bf8f4306de1301301aa961886121329dcf9b11847d823e03`. The draft also failed to inventory an added `lemma` and an added `private theorem`. The draft checker was therefore rejected and the production checker remains byte-identical. This failure does not change a Lean theorem.

The hostile result assumes Python 3.11 or newer with exactly `-I -S -B` and either no optimization or one `-O`; the named checker and theorem bytes; the pinned Lake manifest and Lean release; ordinary stability of the checked filesystem objects during each bounded read; the correctness of Python, Lean, Mathlib, SHA-256, the operating system, and the hardware; and the exact seven-name query inventory. The loader rejects symbolic-link or multiply linked checker/source leaves, double-reads each file through one no-follow descriptor, checks parent and file identities, compiles the captured checker bytes, separates standard output from standard error, requires canonical JSON, restores every changed checker global in `finally`, and rechecks tracked bytes after all cases. These controls reduce the trusted surface; they do not remove it.

6.1.3 Byte and process controls, step by step

Step	Control	Purpose and nonclaim
1	Require Python 3.11 or newer, `-I -S -B`, and optimization level zero or one.	Excludes ambient site packages, ignores Python environment settings, uses a safe import path, prevents bytecode-cache writes, and names the two modes. It does not authenticate Python.
2	Resolve checker, theorem, and harness paths from the harness file.	Removes dependence on the caller's working directory. Resolution is not atomic.
3	`lstat` every lexical parent before reading; require a real directory.	Detects the tested parent substitution and symlink routes. It cannot prevent a coordinated change outside the observation window.
4	`lstat` the leaf; require one regular, non-symbolic, single-linked file.	Rejects leaf symlinks, devices, directories, and hard-link aliases. One link does not prove authenticity.
5	Open read-only with `O_NOFOLLOW`, when supplied by the platform, and close-on-exec.	Narrows a link swap and descriptor inheritance. The operating system remains trusted.
6	Compare device, inode, mode, link count, size, modification time, and change time before, during, and after the read.	Detects bounded changes represented by those fields. It is not a transaction or complete storage monitor.
7	Read one descriptor to end twice, seek between reads, and require equal bytes and length.	Catches ordinary mid-read byte drift and truncation. Equal reads do not establish provenance.

Step	Control	Purpose and nonclaim
8	Recheck every parent identity after the read.	Closes the bounded parent endpoint comparison; it does not monitor the tree continuously.
9	Hash the captured bytes and require the reviewed checker or source SHA-256.	Binds exact bytes; it does not authenticate their producer or reviewer.
10	Compile and execute the captured checker byte string under a digest-derived private module name.	Keeps checked and executed checker representations equal. Python execution remains trusted.
11	Put each mutant in a private temporary directory and rebind only the in-process expected source digest.	Lets the declared change reach its later policy, kernel, or axiom check. The rebind is test machinery, absent from production CI, and gives no production authority.
12	Capture output streams separately; require exact status, an empty unexpected stream, and the expected rejection fragment.	Prevents a crash or unrelated rejection from receiving intended credit. A matching fragment is bounded diagnostic evidence, not full trace proof.
13	Reject duplicate/nonfinite JSON; require final LF, no carriage return, compact sorted reserialization, exactly ten production keys, and expected typed values.	Prevents ambiguous encoding and silent field drift; supports parity. It does not prove the values true.
14	Restore source path, digest, and theorem tuple in `finally`; check the postcondition.	Prevents one case contaminating the next. It is not concurrency isolation.
15	Re-read checker/source, bind the harness digest, require canonical record equality with stable evidence, unload the private module, and compare normal/optimized output.	Detects bounded replay drift, binds the evidence relation, and challenges optimization dependence. It is not an independent checker.

6.1.4 Step-by-step hostile cases

The 19 rows below are disjoint for total accounting. Kernel rejection means that the temporary source reached Lean after its test-only digest rebind and did not compile. Policy rejection means that the raw-token rule rejected the source before Lean. Digest rejection means that the unchanged production digest rejected extra bytes before Lean.

Class	Exact change	Purpose	Required and observed result
Baseline positive	Exact tracked source and seven-name inventory.	Establish that the real gate is runnable before failures are interpreted.	Accepted; seven queries and the exact axiom list.
Semantic negative 1	Replace `Real.log_zpow` by `Real.log_pow`.	The exponent is an integer; the natural-power lemma is not a valid substitute.	Kernel rejection.
Semantic negative 2	Reverse $1 < R$ to $R < 1$ in the positive equivalence.	Positive scaled log means product above one.	Kernel rejection.
Semantic negative 3	Reverse $R < 1$ to $1 < R$ in the negative equivalence.	Negative scaled log means product below one.	Kernel rejection.

Class	Exact change	Purpose	Required and observed result
Semantic negative 4	Use the impossible $R = -1$ branch where the proof needs $R = 1$.	Positivity excludes minus one; confusing the branches destroys the zero proof.	Kernel rejection.
Semantic negative 5	Change reciprocal cancellation from equality to inequality.	For $x > 0$, $\log x + \log(x^{-1}) = 0$ exactly.	Kernel rejection.
Semantic negative 6	Change the retained five-factor product from 1 to 2.	Witness arithmetic must be exact, not approximate.	Kernel rejection.
Semantic negative 7	Remove the premise $0 < R$ from the zero theorem.	The logarithm zero equivalence needs a positive argument.	Kernel rejection.
Semantic negative 8	Rename one queried theorem.	The audit must fail when a required qualified name disappears.	Kernel rejection.
Semantic negative 9	Make the retained theorem depend on `sorryAx`.	A compiling theorem with an extra axiom must not satisfy the permitted basis.	Compiled, then axiom-list rejection.
Raw negative 1	Replace the retained proof by `sorry`.	Direct proof escapes must fail before compilation evidence is credited.	Policy rejection.
Raw negative 2	Change the retained theorem declaration to `axiom`.	A declaration without proof must not be credited.	Policy rejection.
Raw negative 3	Insert quote-valued `Char` definitions, an empty string, and a live unqueried `axiom`.	Concrete counterexample to the discarded scanner.	Policy rejection.
Digest negative 1	Add an unrelated `lemma`, with no digest rebind.	Extra source must not enter the production gate silently.	Digest rejection.
Digest negative 2	Add an unrelated `private theorem`, with no digest rebind.	Declaration modifiers must not bypass source custody.	Digest rejection.
Raw negative 4	Put proof-escape words only in a nested comment and a string.	Record the conservative raw policy; the words are not claimed to be live Lean.	Policy rejection.
Scope positive 1	Re-run the extra `lemma` after a test-only digest rebind.	Show that the seven-name query does not enumerate unrelated declarations.	Accepted with seven queries; scope, not kill credit.
Scope positive 2	Re-run the extra `private theorem` after a test-only digest rebind.	Show the same boundary for a modified declaration form.	Accepted with seven queries; scope, not kill credit.
Known limitation	Shorten the in-memory theorem tuple from seven names to six.	Test whether the checker internally authenticates its mutable query tuple.	Accepted with six queries; zero proof credit.

The suite therefore reports one baseline, nine rejected semantic mutations, six rejected raw or digest controls, two accepted scope probes, and one accepted limitation: 19 cases. The three positive acceptances are the baseline and two scope probes. The accepted limitation is separate and is not converted into positive evidence. Two additional controls show that the reviewed macOS/Arm64 and Ubuntu/x86-64 version strings map to the same portable Lean identity: version 4.33.0, the reviewed commit displayed above, and a `Release` build. The raw target triple remains in each live checker output and log. It is not proof identity and does not establish cross-platform kernel equivalence. Normal and optimized runs emit the same canonical 5,808-byte JSON

record, SHA-256 `01e6e00f72a1ae75aa9f31e148b5685d38b2d82b2477aa8bc55ccfa333ebf84c`.

The production checker proves only the following conditional statement about the fixed files and toolchain: Lean accepted the seven named declarations and each reported exactly the three permitted axioms. The generic mathematics can then be read from the declarations' stated premises. The hostile suite adds bounded sensitivity to the named changes and makes two scope boundaries and one custody limitation observable. Neither artifact proves checker correctness, complete Lean lexical classification, enumeration of every declaration, or rejection of every possible proof fault. Neither formalizes concrete SxPID event extraction, the Möbius lattice, canonical certificate bytes, Rust or binary64 refinement, resource correctness, sampling, estimation, calibration, population validity, or an application. The exact case record is `audit/evidence/sxpid2-exact-log-product-hostile-4.33.0.json`; the rejected route is retained in `claims/SX-CERTIFIED-AVERAGED-PID2-001/failures/lean-exact-log-checker-adjudication-v1.md`.

6.1.5 Verification procedure and interpretation

The sequence is ordered so later evidence is not interpreted after an earlier prerequisite fails.

1. Run the production checker with `python3 -I -S -B`. Acceptance means that the exact tracked source and pinned project passed all seven Lean queries.
2. Run it with one `-O` and compare complete canonical output bytes. This challenges optimization dependence; it is not an independent implementation.
3. Run the hostile suite with `python3 -I -S -B`. Require exactly 19 cases, separate category counts and statuses, and unchanged tracked post-replay bytes.
4. Repeat with one `-O`; require the same 5,808 bytes. Each run itself requires the stable tracked hostile JSON to equal that output; source-state and claim gates also reject an older or differently formatted record.
5. Run the certified-SxPID claim checker and self-test in both isolated modes. They bind the command container, assurance MD/TeX/PDF, catalog projection, and retained authorities. They do not turn the generic theorem into an end-to-end refinement proof.
6. Rebuild with the fixed source-date epoch. The producing-toolchain route requires identical PDF bytes. The cross-toolchain route requires equal extracted text and page geometry, embedded fonts, and no rejected LaTeX diagnostics. Every rendered page is then inspected for layout; visual review does not validate mathematics.
7. Run method-catalog, software-identity, Lean-freeze, release-scope, and current-source-state gates. They preserve repository consistency and historical r14 without inserting this post-r14 evidence retroactively. They neither authenticate Git history nor prove a theorem.

Only the baseline and two declared scope probes are positive acceptances. The nine semantic and six raw or digest cases are negative controls and count only because the declared bad change was rejected for its intended reason. The accepted shortened inventory is a zero-credit limitation. Passing means this bounded contract held for these exact bytes and cases, not general completeness.

6.2 Bounded exact qualification

Every nonzero binary count table with total at most four yields 494 tables and 11,856 coordinates. The exact route checks 5,280 event-nesting obligations, 5,280 local net identities, 1,482 direct-MI identities, 3,952 component-net identities, and 5,928 zeta reconstructions. The products classify 5,886 exact zeros, 5,762 positives, and 208 negatives. All report term products and exact-product decisions agree with the independent derivation.

The fail-closed exact-product self-test kills six semantic source mutations, thirteen resealed certificate mutations, and four structural adversaries, for 23 total. The certificate mutations include positive and negative intervals that touch zero at the endpoint inconsistent with their strict exact-product signs. Two additional sentinel controls establish that the auxiliary checker reaches its per-expression and aggregate admission guards with zero calls to the patched power primitive; these controls are not counted among the 23 rejected adversaries. The boundary checker kills a resealed false sign on Counterexample 4.1. The deterministic evolutionary search uses seed `0x5358504944322026`, total 64, population 96, and 96 generations. It evaluates 5,921 distinct tables while searching for a negative informative or misinformative partial atom and finds none. That is negative search evidence, not a theorem of nonnegativity.

7 Five-lens review

Lens	Conclusion
Definition	The event scan uses the shared-exclusions source disjunction and keyed target intersection for four pinned two-source nodes. It does not replace the redundancy union by an intersection and does not import another PID's atoms or axioms.
Proof	Finiteness, positive exact arguments, positive integer n, count weights, and integer Möbius coefficients are explicit. The sign theorem is complete for the fixed empirical coordinate; scientific adequacy is not a premise or conclusion.
Executable refinement	Rust event terms, exact products, report evidence, and a separately implemented Python reconstruction agree on bounded domains. This is strong bounded evidence, not a universal compiler-level refinement theorem.
Numerical/statistical meaning	Exact product comparison removes transcendental sign ambiguity. It does not provide sampling uncertainty, effect stability, or population validity. A tiny exact sign can be statistically unstable.
Adversarial falsification	Exhaustion found the exact counterexample that invalidated the first completeness assumption; mutations challenge events, weights, Möbius signs, comparison, product evidence, and resource traces. Evolutionary failure to find another class of violation remains incomplete evidence.

8 Positive and negative findings

8.1 Positive findings

- Every fixed empirical two-source cumulative and integer-Möbius atom has the exact form $n^{-1} \log R$.
- Exact zero and sign reduce completely to rational comparison with one; no logarithm is evaluated for that decision.
- The live Rust route is resource-bounded before powering and exposes unavailable decisions rather than silently escalating work.
- Independent product reconstruction, rigorous interval containment, Lean algebra, bounded exhaustion, mutations, and evolutionary search provide distinct evidence layers.

8.2 Negative findings and retained limitations

- Empty canonical terms are not a complete zero criterion. Counterexample 4.1 is a concrete negative result and remains in the qualification corpus.
- Exact arithmetic does not imply atom nonnegativity. The small exhaustive corpus contains 208 negative net coordinates.
- Tolerance-only zero classification is avoidable and loses exact information; 5,886 coordinates in the small corpus are exactly zero.
- Exact rational products can be expensive. The bounded route can abstain, and its abstention is not a sign or zero decision.
- The Lean theorem is generic; concrete event extraction and executable refinement are not yet proved end to end in one kernel.
- The scientific choice and interpretation of SxPID, statistical calibration, continuous estimators, and downstream authority remain outside this theorem.
- Exhaustive and evolutionary searches are bounded. Their negative findings cannot replace general mathematical proofs.

8.3 Historical-receipt replay boundary

The total-eight boundary JSON is both bounded scientific evidence and a historical execution receipt. The former command unconditionally rewrote it. A later build changed the executable and full-certificate digests while leaving the bounded finding unchanged, after which the exact claim-custody projection correctly rejected the altered bytes. That overwrite is retained as a negative process result rather than mistaken for a mathematical failure.

Ordinary replay is now read-only: it validates a fresh certificate, compares a declared stable projection, emits the complete live receipt to standard output, and leaves the tracked receipt byte-identical. The outer evidence projection removes exactly the executable and full-certificate digest bindings. A replacement stable binding hashes the certificate payload after validating its complete outer digest and exact outer, tool-binding, and build-context inventories. It excludes only these four leaves:

1. `payload/tool_binding/runtime_source_manifest_sha256`;
2. `payload/tool_binding/build_context/rustc_verbose_version`;
3. `payload/tool_binding/build_context/build_host`; and
4. `payload/tool_binding/build_context/build_target`.

Lockfile, encoding, profile, cache-policy, distribution-status, arithmetic, coordinate, and claim-boundary fields remain equality-bound. Fifty-one hostile controls fix the literal inventories, vary every excluded and retained class, challenge scientific fields, and reject malformed, missing, extra, and digest-inconsistent structures. The `-update-evidence` option is an explicit reviewed custody transition and is not the ordinary qualification mode.

Beyond those 51 targeted controls, the self-test mutates all 1,236 scalar leaves one at a time. Of 276 boundary-receipt leaves, 274 change the stable projection and equality remains for exactly the declared two outer digests. Of 960 certificate-payload leaves, 956 change the replay projection and equality remains for exactly the declared one source-manifest and three build-environment leaves. This exhausts scalar-leaf sensitivity for the recorded schemas, not structural transformations, future schemas, parser faults, or cryptographic failure.

The two same-host certificates retained for this correction differed only at the runtime source-manifest leaf and the resulting outer payload digest. Both yielded replay projection `11641347ef83ef7262b54d8be4b112dfef38bd61af22dfa9f89e4930d2b9beba`. No second operating system or architecture was executed, so this is a declared variable-field projection contract, not cross-build or cross-platform validation. It is also not source, build, dependency, executable, or portable-semantic identity.

9 Reproduction and evidence map

From the repository root:

```
python3 audit/tools/certified-sxpid/scripts/check-exact-products.py
python3 audit/tools/certified-sxpid/scripts/check-exact-products-self-test.py
python3 audit/tools/certified-sxpid/scripts/check-nonsyntactic-zero-boundary.py
python3 audit/tools/certified-sxpid/scripts/challenge-exact-products.py \
  --output audit/evidence/sxpid2-exact-product-evolutionary-challenge.json
python3 -I -S -B scripts/check-lean-exact-log-product.py
python3 -O -I -S -B scripts/check-lean-exact-log-product.py
python3 -I -S -B scripts/check-lean-exact-log-product-self-test.py
python3 -O -I -S -B scripts/check-lean-exact-log-product-self-test.py
scripts/check-exact-log-product-sxpid2-pdf.sh --exact
```

The machine-readable receipts are

- `audit/evidence/sxpid2-exact-product-qualification.json`;
- `audit/evidence/sxpid2-exact-product-mutation-suite.json`;
- `audit/evidence/sxpid2-exact-product-nonsyntactic-zero-boundary.json`;
- `audit/evidence/certified-sxpid2-boundary-replay-portability-20260728.json`;
- `audit/evidence/sxpid2-exact-product-evolutionary-challenge.json`;
- `audit/evidence/sxpid2-exact-product-lean-check-4.33.0.json`; and
- `audit/evidence/sxpid2-exact-log-product-hostile-4.33.0.json`.

10 Conclusion

The exact product theorem is elementary but decisive: once the finite empirical SxPID2 semantics are fixed, exact sign is rational arithmetic. The important methodological advance is not merely the theorem; it is the retained falsification of the first incomplete zero witness and the strict separation of interval-local evidence from exact multiplicative evidence. The repaired route strengthens numerical assurance while leaving every statistical, scientific, and downstream claim at its proper boundary.

References

[1] A. Makkeh, A. J. Gutknecht, and M. Wibral. Introducing a differentiable measure of pointwise shared information. *Physical Review E*, 103:032149, 2021.

[2] P. L. Williams and R. D. Beer. Nonnegative decomposition of multivariate information. *arXiv:1004.2515*, 2010. This reference supplies historical PID context only; its $I_{\min}$ measure is not used by the exact SxPID2 theorem.